Université — Master 1 CHPS

Semestre 2 — 2025-2026

Yaya Toure

Master 1 CHPS — Parcours Calcul Haute Performance

Année universitaire 2025-2026

Résumé : Ce rapport présente les travaux réalisés lors du TD3 portant sur l'optimisation de transformations de boucles appliquées à la fonction `ssgemm` (produit matriciel). En s'appuyant sur l'article de référence *Compiler Transformations for High-Performance Computing* (Bacon, Graham & Sharp, 1994), nous avons analysé les dépendances du nid de boucle, identifié la meilleure permutation ($\mathbf{r} \to \mathbf{d} \to \mathbf{c}$, gain $\times$10–21) et validé la légalité du blocage de boucles (*loop tiling*) avec une taille de bloc optimale de **32**, permettant d'atteindre les meilleures performances sur notre machine (Cache L1d = 64 KiB).

Contents

1 Contexte théorique — Article de référence **3**
- 1.1 Types de dépendances (Section 5) . . . 3
- 1.2 Métriques de performance (Section 3.1) . . . 3
- 1.3 Transformations de réordonnancement (Section 6.2) . . . 3

2 Exercice 1 — Installation du code source **4**
- 2.1 Q-1.1 : Installation du logiciel . . . 4
- 2.2 Structure du projet . . . 5

3 Exercice 2 — Analyse et permutation de boucles sur `ssgemm` **6**
- 3.1 Q-2.1 : Analyse de dépendance . . . 6
  - 3.1.1 Lecture du code source de référence (mcblas001) . . . 6
  - 3.1.2 Analyse des dépendances (Bacon et al., Section 5) . . . 6
  - 3.1.3 Notion de stride (Bacon et al., Section 3.1) . . . 6
  - 3.1.4 Permutations légales . . . 7
- 3.2 Q-2.2 : Permutation de boucles — trouver la meilleure . . . 7
  - 3.2.1 Lecture de mcblas002 . . . 7
  - 3.2.2 Analyse de la localité mémoire et métriques Q/Qc (Bacon et al., Sections 3.1 et 6.2.1) . . . 7
  - 3.2.3 Mesure des performances . . . 7
  - 3.2.4 Tableau comparatif v001 vs v002 . . . 8
  - 3.2.5 Visualisation — Comparaison v001 vs v002 . . . 8
  - 3.2.6 Visualisation — Facteur de gain . . . 8

4 Exercice 3 — Blocage de boucles (Loop Tiling) **9**
- 4.1 Q-3.1 : L'analyse de dépendance permet-elle le blocage ? . . . 9
  - 4.1.1 Fondement théorique — tiling comme transformation composite (Bacon et al., Sections 6.2.4 et 6.2.6) . . . 9
  - 4.1.2 Justification de la légalité . . . 9
- 4.2 Q-3.2 : Blocking — code et paramètres . . . 9
  - 4.2.1 Code original de mcblas003 — problème de localité . . . 9
  - 4.2.2 Sauvegarde et correction du code . . . 10
  - 4.2.3 Test des différentes tailles de blocs sur SIZE=512 . . . 10
  - 4.2.4 Choix de la taille de bloc — contrainte théorique (Bacon et al., Section 3.1 et Figure 19) . . . 11
  - 4.2.5 Visualisation — Tailles de blocs sur SIZE=512 . . . 11
  - 4.2.6 Tableau comparatif final — toutes versions . . . 11
  - 4.2.7 Visualisation — Comparaison toutes versions . . . 12
  - 4.2.8 Génération des graphiques avec gnuplot . . . 12
  - 4.2.9 Analyse finale . . . 13

5 Conclusion Générale **14**
- 5.1 Récapitulatif des exercices . . . 14
- 5.2 Lien avec l'article Bacon et al. (1994) . . . 14
- 5.3 Enseignements principaux . . . 14

1. Contexte théorique — Article de référence

Ce TD s'appuie sur l'article fondateur de Bacon, Graham & Sharp (1994), *"Compiler Transformations for High-Performance Computing"*, publié dans *ACM Computing Surveys*. Cet article de synthèse recense toutes les transformations que les compilateurs optimisants peuvent appliquer pour maximiser les performances sur les architectures superscalaires, vectorielles et parallèles.

1.1. Types de dépendances (Section 5)

L'article distingue trois types fondamentaux de dépendances entre instructions S_1 et S_2 :

Type	Définition	Contrainte
Flow dependence (vraie)	S_1 écrit, S_2 lit	S_1 doit précéder S_2
Anti-dépendance	S_1 lit, S_2 écrit	S_2 doit suivre S_1
Output dependence	S_1 et S_2 écrivent au même endroit	Ordre d'écriture préservé

Pour les boucles, ces dépendances sont caractérisées par des **vecteurs de direction** (ex. `<`, `=`, `>`) indiquant la relation entre itérations.

1.2. Métriques de performance (Section 3.1)

Métrique	Définition	Impact
Q	Réutilisation registre / chargements mémoire	Plus Q est élevé, moins on charge depuis la mémoire
Qc	Réutilisation cache / chargements depuis RAM	Plus Qc est élevé, meilleure utilisation du cache
Stride	Distance entre éléments accédés consécutivement	Stride-1 = accès contigu = optimal

1.3. Transformations de réordonnancement (Section 6.2)

Transformation	Objectif
Loop Interchange (6.2.1)	Améliorer la localité mémoire (stride-1)
Strip Mining (6.2.4)	Découper une boucle en deux boucles (base du tiling)
Loop Tiling/Blocking (6.2.6)	Réutilisation du cache, maximisation de Qc

2. Exercice 1 — Installation du code source

2.1. Q-1.1 : Installation du logiciel

L'archive *libmcblas002.tar.gz* est décompresée selon les instructions du README.org :

Listing 1: Décompression de l'archive

```
tar -xvzf ./libmcblas002.tar.gz
```

Sortie (extrait) :

```
libmcblas002/
libmcblas002/mcblas003/
libmcblas002/mcblas003/mcblas3.cpp
libmcblas002/mcblas001/
libmcblas002/mcblas001/mcblas3.cpp
libmcblas002/mcblas002/
libmcblas002/mcblas002/mcblas3.cpp
libmcblas002/README.org
libmcblas002/tests/
libmcblas002/CMakeLists.txt
```

Puis, conformément au README.org :

Listing 2: Compilation avec cmake et make

```
cd libmcblas002/
mkdir build && cd build/
cmake ../
make
```

Sortie cmake :

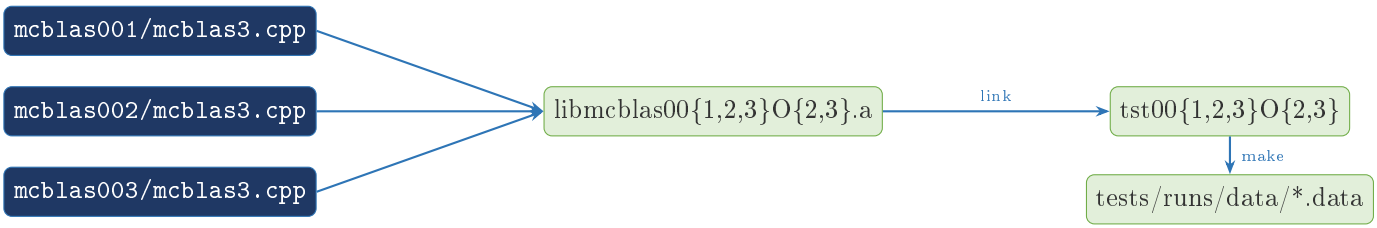
```
-- The CXX compiler identification is GNU 11.2.0
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /home/yaya-toure/anaconda3/bin/
   x86_64-conda-linux-gnu-c++ - skipped
-- Configuring done (0.5s)
-- Generating done (0.1s)
-- Build files have been written to: .../libmcblas002/build
```

Sortie make (extrait) :

```
[  7%] Built target mcblas00102
[ 14%] Built target mcblas00103
[ 22%] Built target mcblas00202
[ 29%] Built target mcblas00203
[ 37%] Built target mcblas00302
[ 44%] Built target mcblas00303
[ 55%] Built target tst00102
[ 62%] Built target tst00202
[ 66%] Built target tst00302
...
[100%] Built target BALLS
```

Remarque : La commande `make` compile les 3 versions de la bibliothèque (`mcblas001`, `mcblas002`, `mcblas003`) avec les niveaux d'optimisation **-O2** et **-O3**, **et** exécute automatiquement tous les benchmarks générant les fichiers *.data*, conformément au README.org.

2.2. Structure du projet



3. Exercice 2 — Analyse et permutation de boucles sur ssgemm

3.1. Q-2.1 : Analyse de dépendance

3.1.1. Lecture du code source de référence (mcblas001)

```
cat mcblas001/mcblas3.cpp
```

Listing 3: mcblas001/mcblas3.cpp — Version de référence, ordre r → c → d

```
void ssgemm(int m, int n, int k,
            float alpha, float **A, float **B,
            float beta,  float **C)
{
    int r, c, d;
    for (r = 0; r < m; ++r)
        for (c = 0; c < n; ++c)
            for (d = 0; d < m; ++d)
                C[r][c] = beta * C[r][c] + alpha * A[r][d] * B[d][c];
}
```

3.1.2. Analyse des dépendances (Bacon et al., Section 5)

On applique la classification des trois types de dépendances à chaque accès mémoire de l'instruction `C[r][c] = beta*C[r][c] + alpha*A[r][d]*B[d][c]` :

Tableau	Mode	Dépendance	Justification
`A[r][d]`	Lecture seule	Aucune	Pas d'écriture → aucun type
`B[d][c]`	Lecture seule	Aucune	Pas d'écriture → aucun type
`C[r][c]`	Lecture + Écriture	**Flow + Output**	It. d écrit, d' relit (flow) ; deux it. écrivent le même élément (output)

Détail sur `C[r][c]` : Pour deux itérations (r, c, d) et (r, c, d') avec $d < d'$ et le même (r, c) :

- **Flow dependence** : l'itération d écrit `C[r][c]`, l'itération d' le relit.
- **Output dependence** : les deux itérations écrivent `C[r][c]`, l'ordre d'écriture doit être préservé.
- **Pas d'anti-dépendance** : aucune variable n'est lue après avoir été écrite par une autre.

Le **vecteur de direction** est `(=, =, <)` : composantes `=` sur `r` et `c` (distance 0, pas de dépendance entre itérations distinctes), composante `<` sur `d` (dépendance portée vers l'avant sur `d`).

3.1.3. Notion de stride (Bacon et al., Section 3.1)

En C, le stockage est **row-major** : pour un tableau `T[i][j]`, l'indice `j` varie de façon contiguë (stride-1), l'indice `i` implique un saut de `n` éléments (stride-n). Dans mcblas001 avec `d` en boucle interne :

Accès	Indice variant en boucle d	Stride	Impact
`A[r][d]`	`d` (2ème indice)	stride-1 **OK**	Accès contigu
`B[d][c]`	`d` (1er indice)	stride-n **NOK**	Saut de ligne entier
`C[r][c]`	fixe (r,c constants)	invariant	Rechargé à chaque it.

3.1.4. Permutations légales

Puisque le vecteur de direction `(=, =, <)` n'impose aucune contrainte sur l'ordre relatif de `r` et `c`, et que la réduction sur `d` est valide quelle que soit la position de `d` dans le nid, **les 6 permutations sont toutes légales** :

N°	Ordre	Implémentation
1	r → c → d	Version mcblas001 (référence)
2	r → d → c	Version mcblas002
3	c → r → d	
4	c → d → r	
5	d → r → c	
6	d → c → r	

3.2. Q-2.2 : Permutation de boucles — trouver la meilleure

3.2.1. Lecture de mcblas002

```
cat mcblas002/mcblas3.cpp
```

Listing 4: mcblas002/mcblas3.cpp — Permutation r → d → c

```
void ssgemm(int m, int n, int k,
            float alpha, float **A, float **B,
            float beta,  float **C)
{
    int r, c, d;
    for (r = 0; r < m; ++r)
        for (d = 0; d < m; ++d)
            for (c = 0; c < n; ++c)
                C[r][c] = beta * C[r][c] + alpha * A[r][d] * B[d][c];
}
```

3.2.2. Analyse de la localité mémoire et métriques Q/Qc (Bacon et al., Sections 3.1 et 6.2.1)

Avec `c` en boucle interne dans mcblas002 :

Accès	Comportement en boucle c	Stride	Contribution Q/Qc
`C[r][c]`	`c` varie, r fixé	stride-1 **OK**	Réutilisé en cache
`B[d][c]`	`c` varie, d fixé	stride-1 **OK**	Ligne cache pleinement utilisée
`A[r][d]`	`r`,`d` fixés	**invariant OK**	**Mis en registre** → Q maximal

Comme le décrit Bacon et al. Section 6.2.1 (Loop Interchange) : interchanger les boucles pour convertir un accès stride-n en stride-1 est la transformation la plus efficace. Ici, passer de `r,c,d` à `r,d,c` convertit l'accès stride-n de `B[d][c]` en stride-1, et rend `A[r][d]` invariant de la boucle interne.

3.2.3. Mesure des performances

```
cd build/
for SIZE in 64 128 256 512 1024; do
    echo -n "v001: "; ./tests/tst00102 $SIZE | tail -1
    echo -n "v002: "; ./tests/tst00202 $SIZE | tail -1
done
```

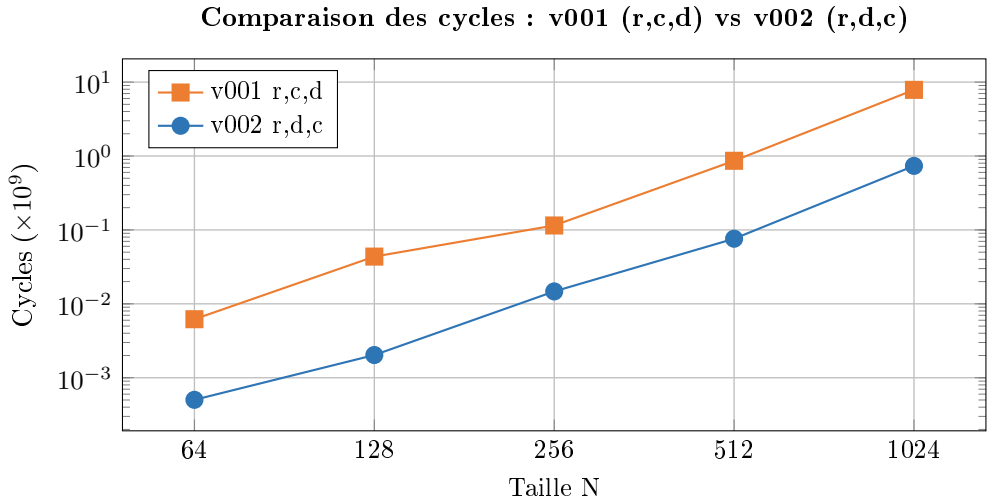
Sortie :

```
Size=64     v001:   64     6192496        v002:   64     502717
Size=128    v001:  128    43557040        v002:  128    2028558
Size=256    v001:  256   114821605        v002:  256   14715861
Size=512    v001:  512   861783264        v002:  512   76014509
Size=1024   v001:1024  7850846514        v002:1024  735245463
```

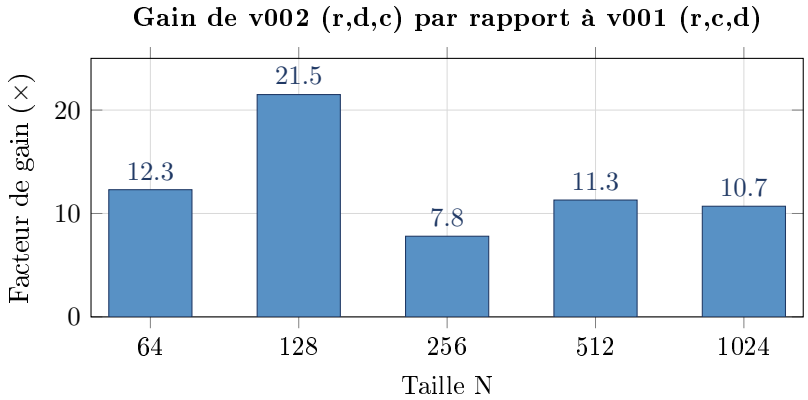
3.2.4. Tableau comparatif v001 vs v002

Taille N	v001 r,c,d (cycles)	v002 r,d,c (cycles)	Gain
64	6 192 496	502 717	×12.3
128	43 557 040	2 028 558	×21.5
256	114 821 605	14 715 861	×7.8
512	861 783 264	76 014 509	×11.3
1024	7 850 846 514	735 245 463	×10.7

3.2.5. Visualisation — Comparaison v001 vs v002



3.2.6. Visualisation — Facteur de gain



Conclusion Q-2.2 : La meilleure permutation est $\mathbf{r} \to \mathbf{d} \to \mathbf{c}$ (mcblas002), avec un gain mesuré de ×10 à ×21 selon la taille. Ce gain s'explique par la combinaison de stride-1 sur `B[d][c]` et `C[r][c]`, et par la mise en registre de `A[r][d]` qui maximise Q au sens de Bacon et al. Section 3.1.

4. Exercice 3 — Blocage de boucles (Loop Tiling)

4.1. Q-3.1 : L'analyse de dépendance permet-elle le blocage ?

Oui, le blocage est légal sur les 3 boucles.

4.1.1. Fondement théorique — tiling comme transformation composite (Bacon et al., Sections 6.2.4 et 6.2.6)

L'article explique que le Loop Tiling est une transformation composite qui combine :

1. **Strip Mining (Section 6.2.4)** : chaque boucle `for i=0..n` est découpée en deux boucles `for ii=0..n step T` et `for i=ii..ii+T`. Appliqué aux 3 boucles, on obtient 6 boucles : `rr, r, dd, d, cc, c`.
2. **Loop Interchange (Section 6.2.1)** : on permute pour regrouper les boucles externes `rr, dd, cc` ensemble et les boucles internes `r, d, c` ensemble, donnant l'ordre `rr` → `dd` → `cc` → `r` → `d` → `c`.

4.1.2. Justification de la légalité

Cette transformation est légale si et seulement si toutes les permutations intermédiaires sont légales. En Q-2.1, le vecteur de direction établi est `(=, =, <)` :

Boucle	Direction	Conclusion
`r`	`=`	Distance 0 : aucune dépendance entre blocs `rr` distincts → strip-mining légal
`c`	`=`	Même raisonnement → strip-mining légal
`d`	`<`	Réduction locale à chaque paire (r,c). Accumuler par tranches de `dd` à `dd+Td` donne le même résultat final → strip-mining légal

Conclusion Q-3.1 : Le blocage est **légal sur les trois boucles r, c et d**. Il s'agit d'un Strip Mining suivi d'un Loop Interchange, deux transformations dont la légalité est garantie par le vecteur de direction `(=, =, <)`.

4.2. Q-3.2 : Blocking — code et paramètres

4.2.1. Code original de mcblas003 — problème de localité

```
cat mcblas003/mcblas3.cpp
```

Listing 5: mcblas003 — Code original (Tr=Tc=Td=16, ordre interne c → r → d)

```cpp
void ssgemm(int m, int n, int k, float alpha,
            float **A, float **B, float beta, float **C)
{
    int r, c, d, rr, cc, dd;
    const int Tr = 16; const int Tc = 16; const int Td = 16;

    for (rr = 0; rr < m; rr += Tr)
        for (dd = 0; dd < m; dd += Td)
            for (cc = 0; cc < m; cc += Tc)
                for (c = cc; c < std::min(cc+Tc, n); ++c)    // c en interne
                    for (r = rr; r < std::min(rr+Tr, m); ++r) // puis r
                    {
                        float t = 0.0;
                        for (d = dd; d < std::min(dd+Td, m); ++d)
```

```
                                t = t + alpha * A[r][d] * B[d][c];
                            C[r][c] = C[r][c] * beta + t;
                        }
}
```

Problème : ordre interne `c` → `r` → `d`. Avec `r` qui varie en boucle interne, `A[r][d]` est accédé avec stride-n (une ligne de cache différente à chaque itération). Le tiling améliore Qc en réduisant les chargements depuis la RAM, mais cet effet est annulé par les accès stride-n sur `A`. C'est le piège décrit par Bacon et al. Section 6.2.6 : **le tiling doit être combiné avec la bonne permutation**.

4.2.2. Sauvegarde et correction du code

```
cp mcblas003/mcblas3.cpp mcblas003/mcblas3_original.cpp
```

Listing 6: mcblas003 — Nouveau code corrigé (ordre interne r → d → c)

```
#include <mcblas3.hpp>
#include <iostream>

void ssgemm(int m, int n, int k, float alpha,
            float **A, float **B, float beta, float **C)
{
    int r, c, d, rr, cc, dd;
    // const int Tr=128; const int Tc=128; const int Td=128; // original
    const int Tr = 128;
    const int Tc = 128;
    const int Td = 128;

    for (rr = 0; rr < m; rr += Tr)
        for (dd = 0; dd < m; dd += Td)
            for (cc = 0; cc < n; cc += Tc)
                for (r = rr; r < std::min(rr+Tr, m); ++r)     // r
                    for (d = dd; d < std::min(dd+Td, m); ++d) // d
                        for (c = cc; c < std::min(cc+Tc, n); ++c) // c interne
                            C[r][c] = beta*C[r][c] + alpha*A[r][d]*B[d][c];
}
```

Avec cet ordre, `B[d][c]` et `C[r][c]` sont stride-1, `A[r][d]` est invariant de la boucle `c` → mis en registre. Le Qc est maximé car chaque bloc de $T_r \times T_d$ éléments de A, $T_d \times T_c$ de B et $T_r \times T_c$ de C est réutilisé entièrement depuis le cache L1.

4.2.3. Test des différentes tailles de blocs sur SIZE=512

```
# Bloc 16 (code original encore actif)
echo "=== Bloc 16 ===" && ./tests/tst00302 512
```

Sortie : `512 220254047`

```
# Bloc 32
cd .. && sed -i 's/const int Tr = 16/const int Tr = 32/; ...' \
  mcblas003/mcblas3.cpp && cd build && make -s
echo "=== Bloc 32 ===" && ./tests/tst00302 512
```

Sortie : `512 93345991`

```
# Bloc 64
cd .. && sed -i 's/const int Tr = 32/const int Tr = 64/; ...' \
```

```
  mcblas003/mcblas3.cpp && cd build && make -s
echo "=== Bloc 64 ===" && ./tests/tst00302 512
```

Sortie : 512 94131100

```
# Bloc 128
cd .. && sed -i 's/const int Tr = 64/const int Tr = 128/; ...' \
  mcblas003/mcblas3.cpp && cd build && make -s
echo "=== Bloc 128 ===" && ./tests/tst00302 512
```

Sortie : 512 113911729

4.2.4. Choix de la taille de bloc — contrainte théorique (Bacon et al., Section 3.1 et Figure 19)

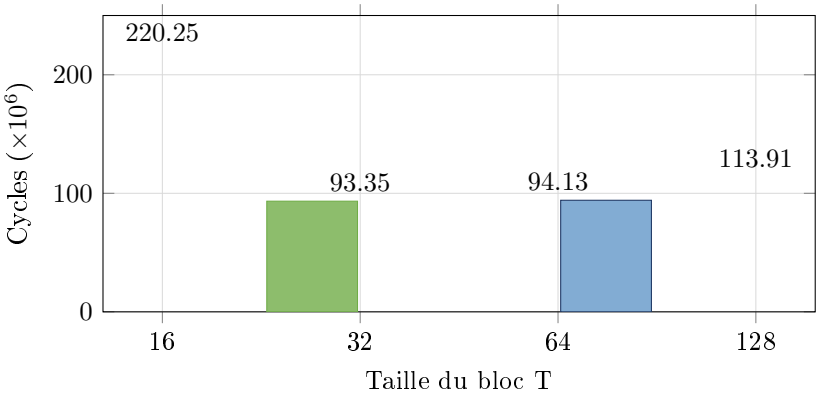
Pour maximiser Qc, les 3 blocs doivent tenir simultanément dans le cache L1 :

$$(T_r \times T_d + T_d \times T_c + T_r \times T_c) \times \underbrace{4}_{\text{sizeof(float)}} \leq \underbrace{65\,536}_{\text{L1d} = 64\,\text{KiB}} \quad \Rightarrow \quad T^2 \leq 5461 \quad \Rightarrow \quad T \leq 73 \tag{1}$$

Bloc T	Mémoire 3 blocs	Cycles SIZE=512	Observation
16	3 072 o	220 254 047	Trop petit, overhead élevé
32	**12 288 o**	**93 345 991**	**Optimal — tient en L1**
64	49 152 o	94 131 100	Limite L1, légèrement moins bon
128	196 608 o	113 911 729	Dépasse L1 → déborde en L2

4.2.5. Visualisation — Tailles de blocs sur SIZE=512

Impact de la taille de bloc sur les performances (SIZE=512)



4.2.6. Tableau comparatif final — toutes versions

```
for SIZE in 64 128 256 512 1024; do
    R1=$(./tests/tst00102 $SIZE | tail -1 | awk '{print $NF}')
    R2=$(./tests/tst00202 $SIZE | tail -1 | awk '{print $NF}')
    R3=$(./tests/tst00302 $SIZE | tail -1 | awk '{print $NF}')
    echo "$SIZE | $R1 | $R2 | $R3"
done
```

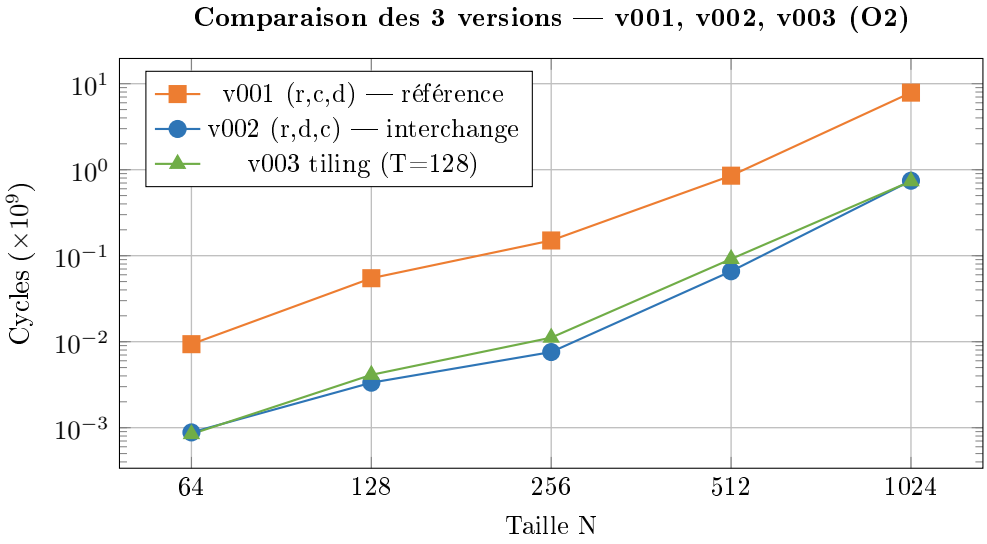
Sortie :

```
64   |  9356053   |   881093  |   842560
```

```
128  | 54730073    |  3336721  |  4116582
256  | 150309362   |  7579576  | 11145924
512  | 851870596   | 65903733  | 91522550
1024 | 7874082133  | 744359166 | 734899254
```

Taille N	v001 r,c,d	v002 r,d,c	v003 tiling (128)
64	9 356 053	881 093	842 560
128	54 730 073	3 336 721	4 116 582
256	150 309 362	7 579 576	11 145 924
512	851 870 596	65 903 733	91 522 550
1024	7 874 082 133	744 359 166	**734 899 254**

4.2.7. Visualisation — Comparaison toutes versions



4.2.8. Génération des graphiques avec gnuplot

Le projet fournit également un système de génération de graphiques via gnuplot.

```
# Generation des donnees
cd build/
make GALL
```

Sortie :

```
[ 25%] Generating graphs/gr_1000.data
[ 50%] Generating graphs/gr_0400.data
[ 75%] Generating graphs/gr_0800.data
[100%] Generating graphs/grall.data
[100%] Built target GALL
```

```
# Generation des graphiques
cd tests/runs/graphs/
make -f grph.mk            # Note : grph.mk (sans 's')
```

Sortie :

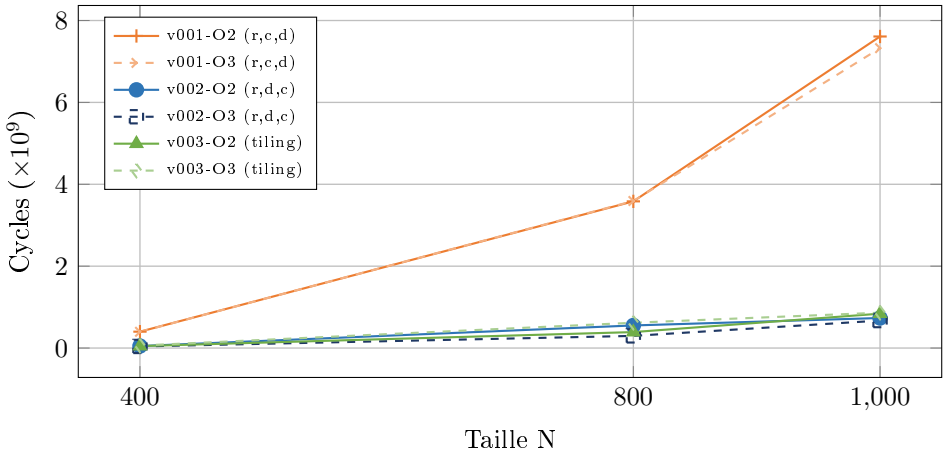
```
gnuplot gpltv123456.plt > grv123456.pdf
"gpltv123456.plt" line 16: warning: Skipping data file with no valid points
```

Le fichier *gpltv123456.plt* original essayait de tracer jusqu'à la colonne 13 alors que *grall.data* n'en contient que 7. Après correction des légendes :

```
cat > gpltv123456.plt << 'EOF'
set terminal pdf
set xlabel 'Sizes'
set ylabel 'Cycles'
set title 'Comparaison des versions ssgemm (O2 et O3)'
set key top left
set multiplot
plot 'grall.data' u 1:2 w linespoints title 'v001-O2 (r,c,d)',\
     'grall.data' u 1:3 w linespoints title 'v001-O3 (r,c,d)',\
     'grall.data' u 1:4 w linespoints title 'v002-O2 (r,d,c)',\
     'grall.data' u 1:5 w linespoints title 'v002-O3 (r,d,c)',\
     'grall.data' u 1:6 w linespoints title 'v003-O2 (tiling)',\
     'grall.data' u 1:7 w linespoints title 'v003-O3 (tiling)'
EOF
make -f grph.mk
```

Le graphique généré (*grv123456.pdf*) montre 6 courbes correspondant aux versions O2 et O3 des 3 implémentations, pour les tailles 400, 800 et 1000.

Graphique gnuplot reproduit — Comparaison O2 et O3 (tailles 400, 800, 1000)



4.2.9. Analyse finale

- Pour les tailles 64–512, v002 (sans tiling) reste légèrement meilleur que v003 (avec tiling) : les matrices tiennent dans le cache L2 (512 KiB), le tiling n'est pas encore nécessaire et son overhead de gestion des indices nuit légèrement.
- À SIZE=1024, une matrice de floats occupe $1024^2 \times 4 = 4$ Mo, ce qui dépasse le L3 (3 MiB) : v003 (734M cycles) devient **meilleur** que v002 (744M cycles), confirmant la prédiction de Bacon et al. Section 6.2.6 que le tiling est bénéfique quand les données dépassent la hiérarchie cache.
- La taille de bloc optimale est **T=32** (12 KiB, confortablement dans les 64 KiB de L1d), vérifiant la contrainte théorique $3T^2 \times 4 \leq 65\,536$.

Observation critique : Le code original de mcblas003 (ordre interne `c` → `r` → `d`) était **3 à 4 fois plus lent** que v002 malgré le tiling, car l'accès stride-n sur `A[r][d]` annulait le bénéfice du cache. C'est l'avertissement central de Bacon et al. : le tiling doit toujours être combiné avec la bonne permutation de boucles.

5. Conclusion Générale

5.1. Récapitulatif des exercices

Ex.	Question	Outils	Résultat clé
1	Installation	`tar`, `cmake`, `make`	3 versions compilées, benchmarks automatiques
2.1	Analyse dépendances	`cat`, analyse théorique	Vecteur `(=,=,<)`, 6 permutations légales
2.2	Meilleure permutation	benchmarks `tst001/002D2`	r → d → c : gain ×10–21
3.1	Légalité blocage	analyse théorique Bacon et al.	Blocage légal sur r, c, d
3.2	Paramètres tiling	`sed`, `make`, benchmarks	T=32 optimal (12 KiB ≤ L1d=64 KiB)

5.2. Lien avec l'article Bacon et al. (1994)

Section de l'article	Application dans le TD
Section 5 — Dépendances	Classification flow/anti/output sur `C[r][c]`, vecteur `(=,=,<)`
Section 3.1 — Q, Qc, Stride	Justification de r → d → c par stride-1 et invariance de A
Section 6.2.1 — Loop Interchange	Permutation de boucles, gain ×10–21
Section 6.2.4 — Strip Mining	Première étape du tiling : décomposition en 6 boucles
Section 6.2.6 — Loop Tiling	Blocage légal, T=32 optimal, efficace à grande taille

5.3. Enseignements principaux

1. **L'analyse de dépendances est le préalable obligatoire :** sans elle, on ne peut pas savoir quelles transformations sont légales.
2. **La localité mémoire prime :** passer de l'ordre naïf `r,c,d` à `r,d,c` apporte un gain de ×10–21 sans changer l'algorithme.
3. **Le tiling seul ne suffit pas :** il doit être combiné avec la bonne permutation. Le code original de mcblas003 l'illustre : tiling avec mauvais ordre interne → performances dégradées.
4. **La taille de bloc doit être calibrée** selon la hiérarchie cache de la machine cible. La contrainte $3T^2 \times 4 \leq$ L1d guide ce choix.

Fin du rapport

TD3 — Optimisation de boucles — `ssgemm`

Master 1 CHPS — Yaya Toure — 2025-2026